

The Ultimate Web Development Handbook



Event Bubbling and Delegation in JavaScript

Understanding **event bubbling** and **event delegation** is essential for writing clean, efficient event-driven code in JavaScript.

What is Event Bubbling?

When an event occurs on a DOM element, it **bubbles up** through its ancestors. This means the event is first captured and handled by the **target element**, and then propagated upward to its parent, grandparent, and so on.

Example:

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
```

```
document.getElementById('child').addEventListener('click', () => {
  console.log('Child clicked');
});

document.getElementById('parent').addEventListener('click', () => {
  console.log('Parent clicked');
});
```

Output when button is clicked:

```
Child clicked
Parent clicked
```

This shows that the event bubbles from the child to the parent.

Stopping Event Bubbling

To stop the event from bubbling up the DOM tree, use:

```
event.stopPropagation();
```

```
button.addEventListener('click', (event) => {  
  event.stopPropagation();  
  console.log('This won't bubble up');  
});
```

What is Event Delegation?

Event delegation is a technique where a single event listener is added to a **common parent** instead of individual child elements. This works because of event bubbling.

Why Use Event Delegation?

- Improved performance with many child elements
- Simplified code
- Useful for dynamic elements added after page load

Example:

```
<ul id="menu">  
  <li>Home</li>  
  <li>About</li>  
  <li>Contact</li>  
</ul>
```

Instead of attaching a click listener to every `li`, delegate it to the `ul`:

```
document.getElementById('menu').addEventListener('click', (event) => {  
  if (event.target.tagName === 'LI') {
```

```
    console.log(`You clicked on ${event.target.textContent}`);  
  }  
});
```

This even works if new `<li>` elements are added later using JavaScript.

Summary

- **Event bubbling:** Events move up the DOM tree from child to parent.
- Use `event.stopPropagation()` to prevent bubbling.
- **Event delegation:** Handle events at a parent level for better performance and maintainability.

Preventing Default Behavior in JavaScript

Many HTML elements have **default behaviors**. For example:

- Clicking a link navigates to a new URL
- Submitting a form reloads the page
- Pressing certain keys triggers built-in browser actions

In JavaScript, you can **override** these default behaviors using `event.preventDefault()`.

Syntax

```
element.addEventListener('eventType', (event) => {  
    event.preventDefault();  
});
```

This method tells the browser **not to perform** the default action associated with the event.

Common Use Cases

1. Preventing Form Submission

By default, submitting a form reloads the page. You can prevent this to handle form data using JavaScript:

```
<form id="myForm">  
  <input type="text" />
```

```
<button type="submit">Submit</button>
</form>
```

```
const form = document.getElementById('myForm');

form.addEventListener('submit', (event) => {
  event.preventDefault();
  console.log('Form submission prevented');
});
```

2. Preventing Link Navigation

You may want to handle navigation manually using JavaScript:

```
<a href="https://codewithharry.com" id="myLink">Go to CodeWithHarry</a>
```

```
const link = document.getElementById('myLink');

link.addEventListener('click', (event) => {
  event.preventDefault();
  console.log('Navigation prevented');
});
```

3. Preventing Checkbox Default Toggle

In special cases, you might want to keep a checkbox from changing state:

```
const checkbox = document.querySelector('input[type="checkbox"]');

checkbox.addEventListener('click', (event) => {
  event.preventDefault();
  console.log('Checkbox toggle prevented');
});
```

When Not to Use `preventDefault()`

Avoid using `preventDefault()` unless necessary. Overusing it can confuse users and break expected behavior (like tabbing between fields or pressing Enter to submit).

Summary

- Use `event.preventDefault()` to cancel default browser actions.
- Common in form handling, link overrides, or input control.
- Always test to ensure usability and accessibility are not harmed.

Working with `localStorage` in JavaScript

The `localStorage` API allows you to store key-value data in the browser, with no expiration date. This data persists across page reloads and browser sessions, unless manually cleared.

Key Features

- Stores data as **strings**
- Synchronous API (blocking)
- Maximum storage capacity: around 5MB (varies by browser)
- Data is domain-specific

Basic Syntax

```
// Set item
localStorage.setItem('key', 'value');

// Get item
const value = localStorage.getItem('key');

// Remove item
localStorage.removeItem('key');

// Clear all items
localStorage.clear();
```


Example: Storing and Retrieving Data

```
localStorage.setItem('username', 'haris');

const user = localStorage.getItem('username');
console.log(user); // Output: haris
```

Storing Objects

Since `localStorage` only stores strings, you must convert objects using `JSON.stringify()` and retrieve them using `JSON.parse()` :

```
const user = {
  name: 'Harry',
  age: 25
};

// Store the object
localStorage.setItem('user', JSON.stringify(user));

// Retrieve the object
const storedUser = JSON.parse(localStorage.getItem('user'));
console.log(storedUser.name); // Output: Harry
```

Checking if Key Exists

```
if (localStorage.getItem('theme')) {
  console.log('Theme is set');
}
```

Use Cases

- Saving user preferences (e.g., theme, language)

- Caching small amounts of data
- Keeping users logged in (store tokens)
- Temporarily saving form data

Limitations

- Synchronous (can block main thread)
- Storage limit (~5MB)
- No automatic expiration (unlike `sessionStorage` or cookies)
- Not secure for sensitive data (easily inspectable via DevTools)

Summary

- `localStorage` provides simple, persistent browser storage.
- Use `setItem`, `getItem`, `removeItem`, and `clear` methods.
- Always stringify objects before storing.
- Avoid storing sensitive data.

Parsing JSON in JavaScript

JSON (JavaScript Object Notation) is a lightweight data format used for exchanging data between a server and a client. JavaScript provides built-in methods to **parse** and **stringify** JSON.

What is JSON?

JSON is a **string representation** of data structured in key-value pairs.

Example JSON string:

```
{
  "name": "Harry",
  "age": 25,
  "skills": ["JavaScript", "Python"]
}
```

This looks like a JavaScript object, but it's actually a string.

Parsing JSON with `JSON.parse()`

To convert a JSON string into a JavaScript object, use `JSON.parse()` :

```
const jsonString = '{"name":"Harry","age":25}';

const user = JSON.parse(jsonString);
console.log(user.name); // Output: Harry
```

Note:

- The string **must be valid JSON**, otherwise `JSON.parse()` will throw an error.
 - All keys must be enclosed in double quotes (`"`).
-

Stringifying JavaScript Objects with `JSON.stringify()`

To convert a JavaScript object into a JSON string, use `JSON.stringify()` :

```
const user = {  
  name: 'Harry',  
  age: 25  
};  
  
const json = JSON.stringify(user);  
console.log(json); // Output: {"name":"Harry","age":25}
```

Common Use Case: `localStorage`

Since `localStorage` can only store strings, you often use JSON methods to store and retrieve objects:

```
// Store object  
localStorage.setItem('user', JSON.stringify(user));  
  
// Retrieve and parse object  
const storedUser = JSON.parse(localStorage.getItem('user'));
```

Handling Errors

Wrap parsing in a `try...catch` block to handle invalid JSON:

```
try {  
  const data = JSON.parse(badJSONString);  
} catch (error) {  
  console.error('Invalid JSON:', error.message);  
}
```

Summary

- Use `JSON.parse()` to convert a JSON string into a JavaScript object.
- Use `JSON.stringify()` to convert a JavaScript object into a JSON string.
- These methods are commonly used with `localStorage`, APIs, and data exchange.

Error Handling in JavaScript

Errors are inevitable in any application. JavaScript provides robust tools to **detect**, **handle**, and **respond** to runtime errors, allowing your code to fail gracefully.

Types of Errors

- **Syntax Errors:** Mistakes in the code structure (e.g., missing brackets)
 - **Runtime Errors:** Occur during execution (e.g., accessing undefined variables)
 - **Logical Errors:** Code runs but doesn't behave as intended
-

The `try...catch` Statement

Use `try...catch` to handle errors without crashing the entire script.

Syntax:

```
try {  
    // Code that may throw an error  
} catch (error) {  
    // Handle the error  
}
```

Example:

```
try {  
    let result = 10 / x; // x is not defined  
} catch (error) {  
    console.log('An error occurred:', error.message);  
}
```

The `finally` Block

The `finally` block is optional and always runs, **whether an error occurred or not**.

```
try {  
    // Risky code  
} catch (error) {  
    // Handle error  
} finally {  
    // Always runs  
    console.log('Cleanup complete');  
}
```

Throwing Custom Errors

You can manually throw errors using the `throw` statement.

```
function divide(a, b) {  
    if (b === 0) {  
        throw new Error('Cannot divide by zero');  
    }  
    return a / b;  
}  
  
try {  
    divide(5, 0);  
} catch (error) {  
    console.error(error.message); // Output: Cannot divide by zero  
}
```


Catching Specific Error Types

The `catch` block receives an `Error` object with useful properties:

```
try {  
  JSON.parse('invalid JSON');  
} catch (error) {  
  console.log(error.name);    // SyntaxError  
  console.log(error.message); // Unexpected token i in JSON  
}
```

Best Practices

- Always handle expected errors (e.g., API failures, user input)
 - Avoid swallowing errors silently — log them
 - Use specific error messages for debugging
 - Don't use try-catch for control flow in performance-critical code
-

Summary

- Use `try...catch` to prevent your app from crashing.
- Add `finally` for cleanup logic.
- Use `throw` to raise custom errors.
- Proper error handling makes applications more stable and user-friendly.

Timers and Intervals in JavaScript

JavaScript provides built-in functions to schedule code execution after a delay or at repeated intervals. These are useful for animations, delayed actions, auto-saving, and more.

`setTimeout()` : Run Code After a Delay

The `setTimeout()` function executes code **once** after a specified delay (in milliseconds).

Syntax

```
setTimeout(callback, delay);
```

Example

```
setTimeout(() => {  
  console.log('This runs after 2 seconds');  
}, 2000);
```

Canceling a Timeout

Use `clearTimeout()` to stop a scheduled timeout.

```
const timeoutId = setTimeout(() => {  
  console.log('Will not run');  
}, 3000);  
  
clearTimeout(timeoutId);
```

setInterval() : Run Code Repeatedly

The `setInterval()` function executes code **repeatedly** at a specified interval (in milliseconds).

Syntax

```
setInterval(callback, interval);
```

Example

```
setInterval(() => {  
  console.log('Runs every second');  
}, 1000);
```

Canceling an Interval

Use `clearInterval()` to stop the repetition.

```
const intervalId = setInterval(() => {  
  console.log('Repeating...');  
}, 1000);  
  
setTimeout(() => {  
  clearInterval(intervalId);  
  console.log('Interval stopped');  
}, 5000);
```

Use Cases

- Delayed popups or notifications
- Auto-refreshing data (like a clock or stock ticker)

- Game loops and animations
 - Debouncing or throttling user input
-

Important Notes

- Delays are **not guaranteed** to be exact — they depend on the event loop and execution stack.
 - Avoid overly frequent intervals (`<10ms`) as it may block the main thread.
-

Summary

Function	Purpose	Cancel with
<code>setTimeout()</code>	Run code once after delay	<code>clearTimeout()</code>
<code>setInterval()</code>	Run code repeatedly with delay	<code>clearInterval()</code>